Guía de estudio — TP3, Ejercicio 4

Planificación Round Robin. Sistemas de Tiempo Real

1. Los números de memoria

Tarea	C (unid.)	T (unid.)	C (ms)	T (ms)
A	2	12	128	768
B	2	24	128	1536
C	2	6	128	384
D	1	3	64	192

Escala: 1 unidad = 4 ticks = **64 ms**. Hiperperíodo $\text{lcm}(12, 24, 6, 3) = 24$ unidades = **1536 ms**, con 22 unidades ocupadas y 2 ociosas.

Utilización: $U = 2/12 + 2/24 + 2/6 + 1/3 =$ **0,9167**.

Cuanto: 16 ms nominal, **16,05 ms** medido. Vuelta completa de las cuatro: **63,99 ms** $= 4 \times 16$.

Resultados: activaciones por reporte +4, +2, +8, +16.

Tarea	$n \cdot C$ teórico	`fin1`	$R_{\text{máx}}$	Jitter	¿Pierde?
D	256	**257**	257	11	sí
A	512	**508**	508	1	no
B	512	**524**	524	1	no
C	512	**529**	529	1	sí

2. La historia, en seis pasos

1. En el Ejercicio 3 se midió que en Proteus **el contador de ticks no avanza mientras una tarea consume CPU** (columna `dtT` = 0). El tick mide tiempo ocioso, no tiempo real.
2. Con una sola tarea eso se compensaba. Acá no: con $U = 0{,}9167$ solo 2 de cada 24 unidades son ociosas, así que el reloj del sistema correría **doce veces más lento** que el tiempo real.
3. Se descartó modificar la biblioteca. Entonces el banco se rediseñó para no depender del tick.
4. **Reloj propio:** un contador avanza 1 por cada milisegundo de CPU consumido, dentro del lazo calibrado. La tarea Idle consume el ocio de la misma forma, así que la suma mide tiempo real.
5. **Cuanto:** lo fuerza la tarea con `taskYIELD()`, que termina en `vTaskSwitchContext()` y rota la lista de listos igual que lo haría el tick.
6. **Activaciones:** las libera un despachador con notificaciones de tarea, llamado al cerrar cada cuanto, con el planificador suspendido para que las cuatro arranquen en el mismo instante.

3. Las dos cuentas que hay que saber hacer en el pizarrón

Tiempo de respuesta bajo reparto equitativo. Con n tareas listas, cada una avanza a $1/n$ de velocidad, así que tarda $R \approx n \cdot C$. Con $n = 4$: D tarda $4 \times 64 = 256$ y las otras $4 \times 128 = 512$.

Condición de cumplimiento.

$$n \cdot C_i \le T_i \qquad \Longleftrightarrow \qquad \frac{C_i}{T_i} \le \frac{1}{n}$$

La porción equitativa es 25 %. A pide 16,7 % y B pide 8,3 %: cumplen. C y D piden 33,3 %: pierden.

4. Preguntas probables y respuesta corta

- **¿Tocaste la biblioteca?** No. Los 25 archivos conservan la fecha del paquete, no hay rastro de Timer1 ni CTC, y `portUSE_WDTO` sigue en `WDTO_15MS`. Todo está en `main.cpp` y en dos banderas de `platformio.ini`.
- **¿Entonces esto es Round Robin de verdad?** La rotación sí: `taskYIELD()` llama a `vTaskSwitchContext()`, que toma el siguiente de la lista de listos, exactamente lo mismo que hace el tick con `configUSE_TIME_SLICING`. Lo que cambia es quién dispara el cambio de contexto: la tarea en vez de la interrupción.
- **¿Sigue siendo expropiativo?** Entre prioridades distintas sí: la tarea Idle es expropiada en cuanto se libera una periódica. Entre tareas de igual prioridad el reparto es cooperativo y lo fuerza la tarea.
- **¿Por qué el cuanto vale 16 ms?** Para que sea el mismo valor que `portTICK_PERIOD_MS` y el cronograma sea comparable con el que daría un reparto por expropiación del tick.

- **¿Por qué se pierden plazos si $U < 1$?** Porque $U < 1$ garantiza que el trabajo entra, no cuándo entra. Round Robin reparte en partes iguales e ignora los plazos: C y D piden un tercio y reciben un cuarto.
- **¿Cuál es la más perjudicada?** C. Las dos perjudicadas pierden un plazo por hiperperíodo, pero C se activa 4 veces por hiperperíodo y D se activa 8: en proporción C pierde el 25 % de sus activaciones y D el 12,5 %.
- **¿Por qué $R_{\text{máx}}$ coincide con `fin1`?** Porque el peor caso ocurre en la primera ráfaga, con las cuatro tareas liberadas juntas. Es el *instante crítico* del análisis de planificabilidad, observado experimentalmente.
- **¿Por qué las cuatro compiten todo el tiempo?** D se reactiva a los 192 ms y recién termina su primera ráfaga a los 257: nunca sale de la lista de listos. Por eso A, B y C tardan 4×128 y no 3×128.
- **¿Por qué D tiene 11 ms de jitter y las otras 1?** El despacho ocurre al cerrar cada cuanto, así que una activación puede atrasarse hasta un cuanto. D se activa ocho veces por hiperperíodo, de modo que es la que más veces cae en medio de un cuanto.
- **¿Cómo sabés que el reloj propio mide bien?** Las activaciones crecen +4, +2, +8, +16 entre reportes separados por dos hiperperíodos, que es exactamente lo que corresponde. Si el reloj derivara, los cuatro incrementos se desviarían.
- **¿Cómo detectás un plazo perdido?** Al liberar una activación, si la anterior sigue marcada como pendiente es que no terminó antes de la siguiente. Como $D = T$, eso es exactamente un plazo perdido.
- **¿Por qué el trabajo de fondo está en `loop()`?** Porque la biblioteca ejecuta `loop()` desde `vApplicationIdleHook()`, o sea desde la tarea Idle, que ya es la de menor prioridad y ya tiene su pila. Crear una sexta tarea agotaba el heap del ATmega328P.
- **¿Sirve la cota de Liu y Layland acá?** No. Esa cota es para prioridades fijas asignadas por período (Rate Monotonic). Round Robin no usa prioridades, así que la cota no dice nada sobre este caso. Lo que aplica es $C_i/T_i \leq 1/n$.

5. Las tres cosas que no hay que decir mal

- El tick **no mide tiempo real** en este simulador: mide tiempo ocioso. Ese es el hecho medido del que sale todo el diseño.
- El cuanto **no lo produce la interrupción del tick**. Coincide con el período del tick porque se eligió así, no porque lo genere.
- $U < 1$ **no garantiza** el cumplimiento de plazos bajo cualquier política. Solo dice que el trabajo total cabe en el procesador.

6. Cómo mostrarlo, en un minuto

1. Compilar y correr. La terminal arranca con `unidad=64ms cuanto=16ms politica=RR` y `RAM libre=787`: ahí quedan fijadas la escala y que entran las cinco tareas sin comentar nada.
2. Esperar dos bloques de reporte. Señalar que las activaciones suben +4, +2, +8, +16, que A y B tienen cero plazos perdidos y que C y D pierden.
3. En el analizador, cursores sobre un tramo de ejecución: **16,05 ms**. Sobre una vuelta completa: **63,99 ms**.
4. Cerrar con la columna `fin1`: 257 para D y 508, 524 y 529 para las otras tres, contra los 256 y 512 que predice $R = n \cdot C$.

7. Qué esperar en el Ejercicio 5 (FIFO)

Se cambia una sola línea, `USAR_ROUND_ROBIN` a 0, y desaparecen los `taskYIELD()`: las ráfagas pasan a ser atómicas.

Con las cuatro activadas en $t = 0$ y sin reparto, cada una corre hasta terminar, de modo que `fin1` debería dar aproximadamente **128, 256, 384 y 448 ms**, según el orden en que se las atienda. La primera atendida mejora muchísimo respecto de Round Robin y la última empeora; el instante en que termina la última es el mismo en las dos políticas, porque el trabajo total no cambió.

Para el inciso 5(c), el orden de atención no es estrictamente el de llegada: `vListInsertEnd` inserta en la lista circular de listos *delante del índice actual*, y `listGET_OWNER_OF_NEXT_ENTRY` avanza ese índice en cada selección. El recorrido es cíclico sobre la lista, no una cola FIFO por orden de llegada.